

Source Code :(LED Blinking)

```
void main() {  
  
    TRISB = 0x00; // Configure all PORTB pins as output  
    PORTB = 0x00; // Initially turn OFF all LEDs  
  
    while(1)  
    {  
        // Turn ON LED at RB0  
        PORTB.F0 = 1;  
        PORTB.F1 = 0;  
        PORTB.F2 = 0;  
        PORTB.F3 = 0;  
        Delay_ms(300);  
  
        // Turn ON LED at RB1  
        PORTB.F0 = 0;  
        PORTB.F1 = 1;  
        PORTB.F2 = 0;  
        PORTB.F3 = 0;  
        Delay_ms(300);  
  
        // Turn ON LED at RB2  
        PORTB.F0 = 0;  
        PORTB.F1 = 0;  
        PORTB.F2 = 1;  
        PORTB.F3 = 0;  
        Delay_ms(300);  
  
        // Turn ON LED at RB3  
        PORTB.F0 = 0;  
        PORTB.F1 = 0;  
        PORTB.F2 = 0;  
        PORTB.F3 = 1;  
        Delay_ms(300);  
    }  
}
```

Circuit diagram:

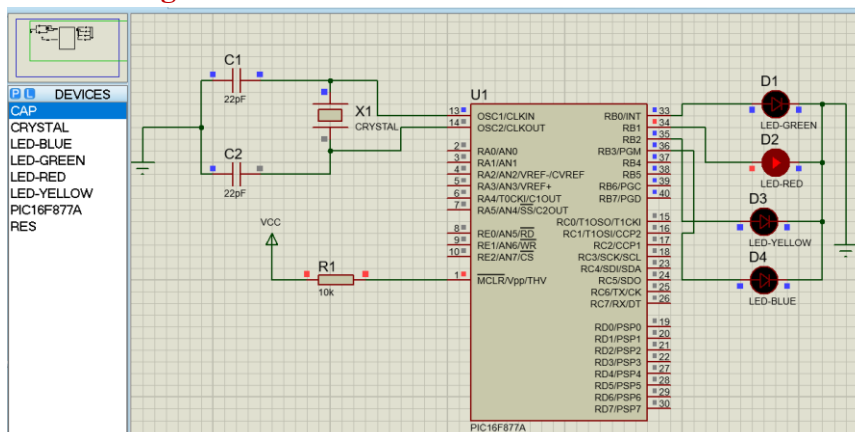


Figure-1: LED flasher using PIC microcontroller.

Source Code :(Seven Segment Display)

```
char array[]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
int i;
void main() {

    TRISB=0x00;
    portb=0x00;
    while(1)
    {
        for(i=0;i<10;i++)
        {
            portb=array[i];
            delay_ms(1000);
        }
    }
}
```

Circuit Diagram:

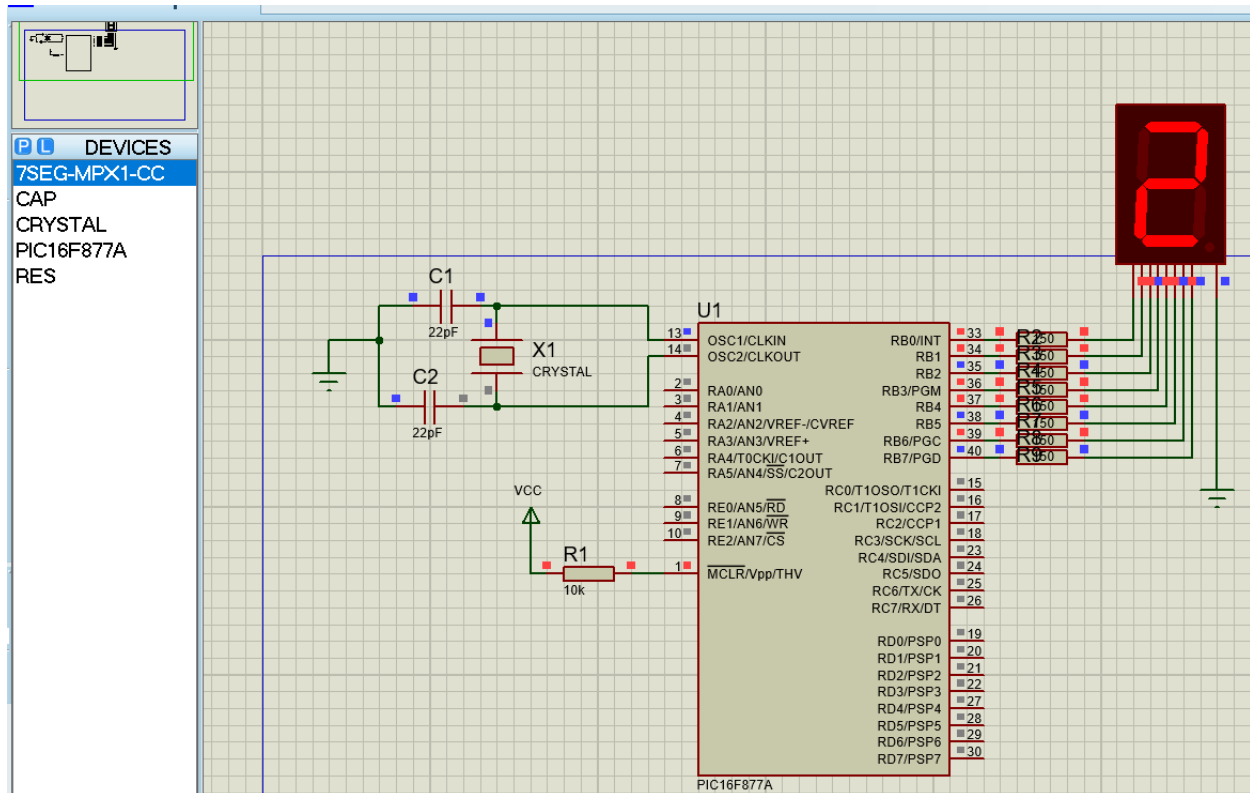


Figure-2: count 0 to 9 in 7 segment display.

```
char array[]={0x3f, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
```

```
void main()
```

 $\{$

```
int i=0;
```

```
int j, Left_digit=0, Right_digit=0, count;
```

```
TRISB=0x00;
```

```
TRISC=0x00;
```

```
portb=0x00;
```

```
portc=0x00;
```

```
while(1)
```

 $\{$

```
Left_digit=i/10;
```

```
Right_digit=i%10;
```

```
for(j=0;j<50;j++)
```

 $\{$

```
portb=array[Left_digit];
```

```
portc.f0=0;
```

```
delay_ms(10);
```

```
portc.f0=1;
```

```
portc.f1=0;
```

```
portb=array[Right_digit];
```

```
delay_ms(10);
```

```
portc.f1=1;
```

}

```
if(i>99)
```

```
i=0;
```

else

```
    i++;
```

$$\}} \quad$$

Circuit Diagram:

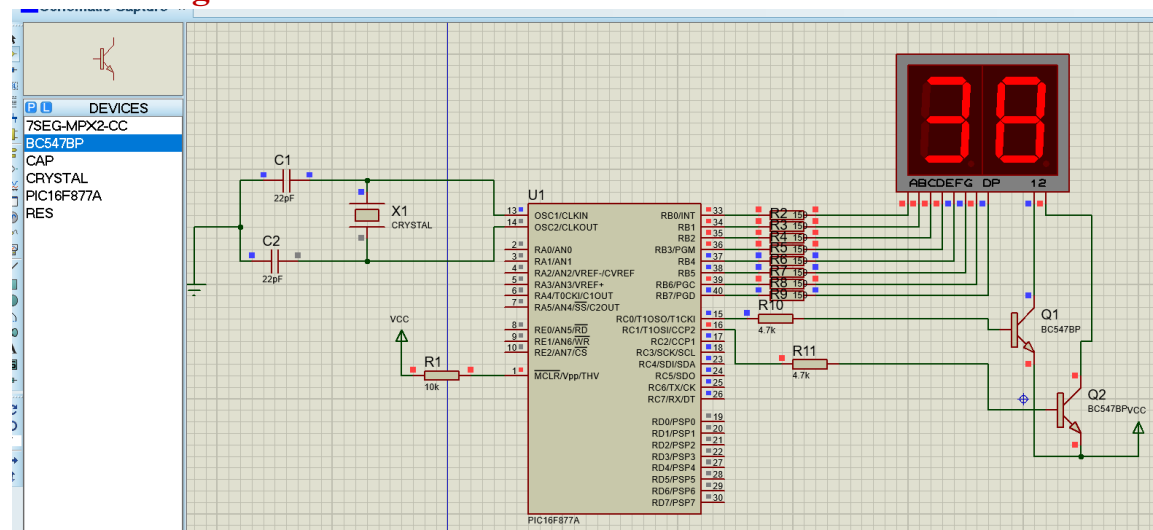


Figure-3: 7-segment multiplexer.

Source Code :

```
char arr[] = {0x40,0x79,0x24,0x30,0x19,0x12,0x02,0x78,0x00,0x10};
```

```
void main() {  
    int count=0,left,right;
```

```
    TRISB = TRISC = 0x00;  
    TRISD = 0xff;  
    PORTB = PORTC = PORTD = 0x00;
```

```
    while(1){  
        //increase button  
        if(PORTD.F0 == 1){  
            Delay_ms(30);    // debounce  
            if(PORTD.F0 == 1){  
                count++;  
                if(count == 100) count = 0;  
  
                while(PORTD.F0 == 1); // wait till release  
            }  
        }
```

```
        //decrease button  
        if(PORTD.F1 == 1){  
            Delay_ms(30);    // debounce  
            if(PORTD.F1 == 1){  
                if(count == 0) count = 0;  
                else count--;  
                while(PORTD.F1 == 1); // wait till release  
            }  
        }
```

```
        //reset button  
        if(PORTD.F2 == 1){  
            delay_ms(30);  
            if(PORTD.F2 == 1) {  
                count = 0;  
            }  
        }
```

```
        left = count/10;  
        right = count%10;
```

```
        PORTC.F0 = 1;  
        PORTB = arr[left]; //segment show  
        Delay_ms(5);  
        PORTC.F0 = 0;
```

```
        PORTC.F1 = 1;
```

```

PORTB = arr[right];
Delay_ms(5);
PORTC.F1 = 0;
}
}

```

Circuit Diagram:

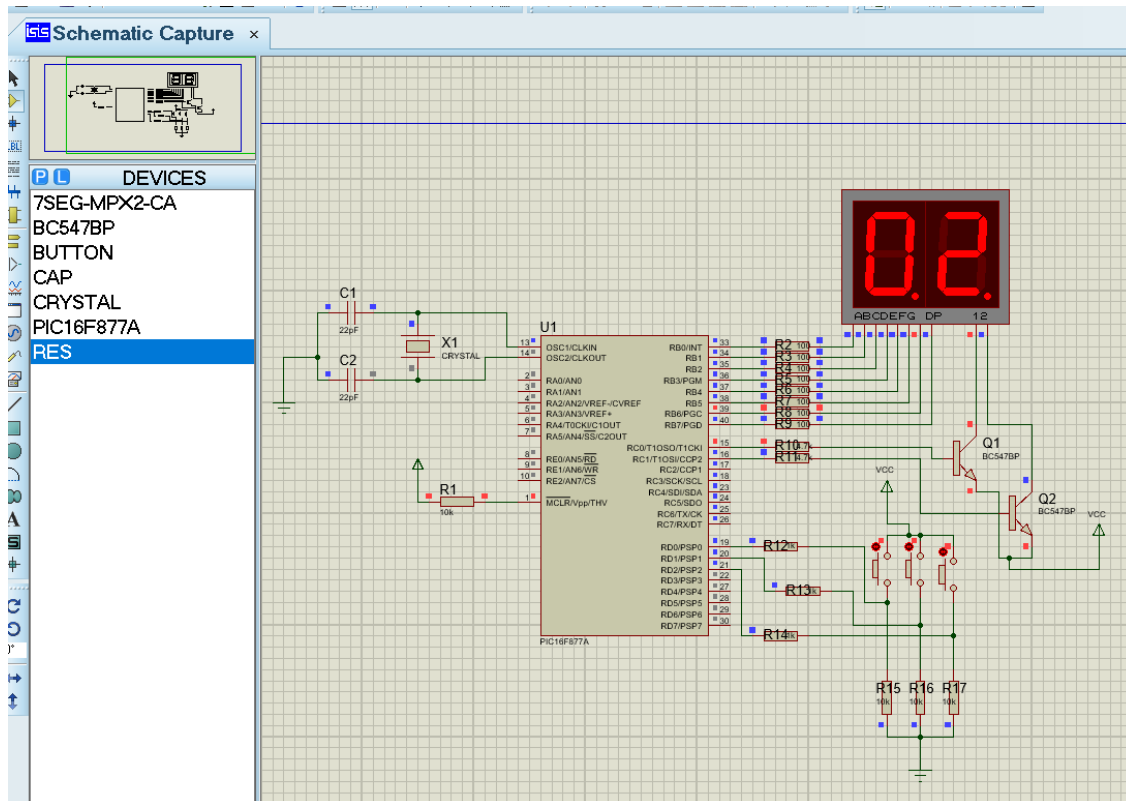


Figure-4: interface double digit 7 segment LED display using push button.

Source Code :

```
void main() {  
    TRISB=0x00; //set port b as output  
    portb=0x00; //initialize as 0  
    while(1)  
    {  
        portb.f0=1;  
        delay_ms(1000);  
        portb.f0=0;  
        delay_ms(1000);  
    }  
}
```

Circuit Diagram:

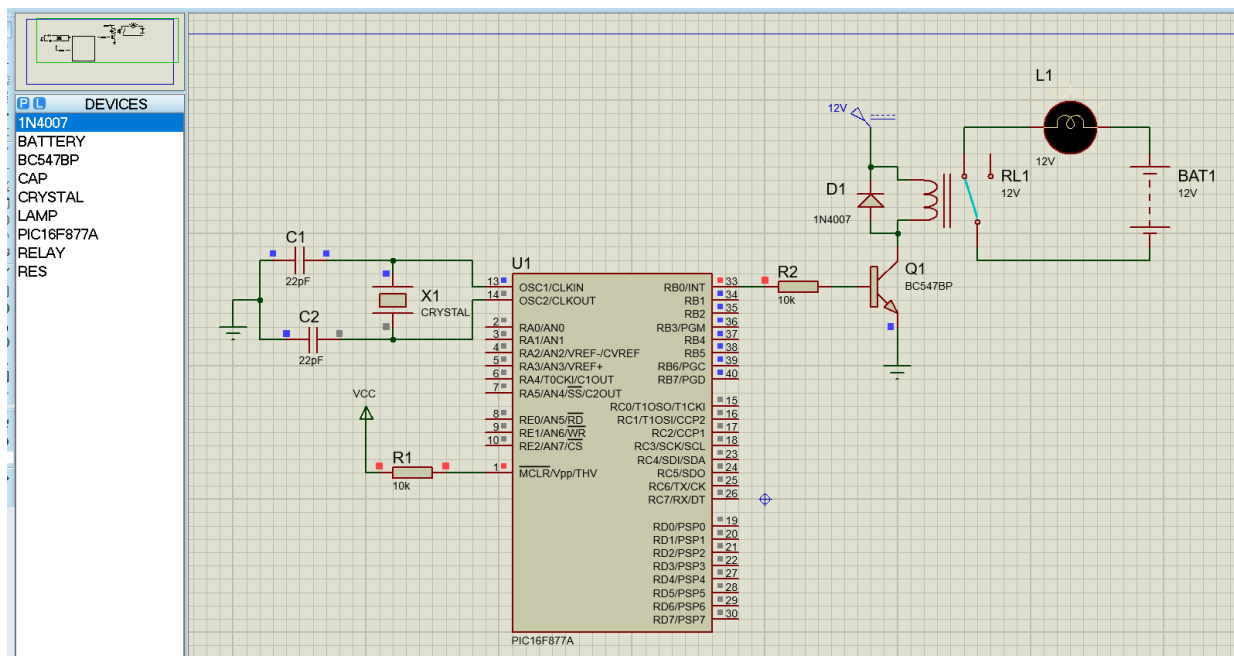


Figure-5: control a high voltage load using mechanical relay.

Source Code :

```
void main()
{
    short duty = 0;
    TRISD = 0xFF;
    TRISB = 0x00;
    TRISC.F2 = 0;
    PORTB.F0 = 1;
    PORTB.F1 = 0;
    PWM1_Init(1000);
    PWM1_Start();
    PWM1_Set_Duty(0);
    while(1)
    {
        // -- Speed UP (RD0) --
        if(PORTD.F0 == 1)
        {
            Delay_ms(50);
            if(PORTD.F0 == 1)
            {
                if(duty <= 245)
                {
                    duty += 10;
                    PWM1_Set_Duty(duty);
                    while(PORTD.F0 == 1); // wait release
                }
            }
        }
        // -- Speed DOWN (RD1) --
        if(PORTD.F1 == 1) // ? PORTD.F1 explicitly
        {
            Delay_ms(50);
            if(PORTD.F1 == 1)
            {
                if(duty > 0) // ? duty > 0 check
                {
                    duty -= 10;
                }
                else
                {
                    duty = 0; // ? ??? negative ??? ??
                }
                PWM1_Set_Duty(duty);
                while(PORTD.F1 == 1); // wait release
            }
        }
    }
}
```

Circuit Diagram:

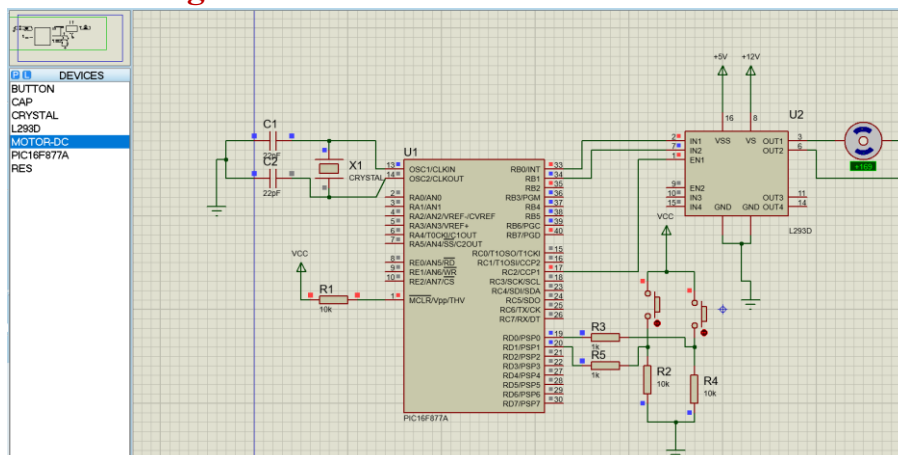


Figure-6: DC motor speed using PIC microcontroller

Source Code :

```
int i;
void main() {
    TRISB.F0 = 0; // RB0 output
    PORTB.F0 = 0;
    while(1){
        // 0 degree
        for(i=0; i<50; i++){
            PORTB.F0 = 1;
            Delay_us(1000); // 1 ms pulse
            PORTB.F0 = 0;
            Delay_us(19000); // total 20 ms
        }
        Delay_ms(500);

        // 90 degree
        for(i=0; i<50; i++){
            PORTB.F0 = 1;
            Delay_us(1500); // 1.5 ms pulse
            PORTB.F0 = 0;
            Delay_us(18500);
        }
        Delay_ms(500);

        // 180 degree
        for(i=0; i<50; i++){
            PORTB.F0 = 1;
            Delay_us(2000); // 2 ms pulse
            PORTB.F0 = 0;
            Delay_us(18000);
        }
        Delay_ms(500);
    }
}
```

Circuit Diagram:

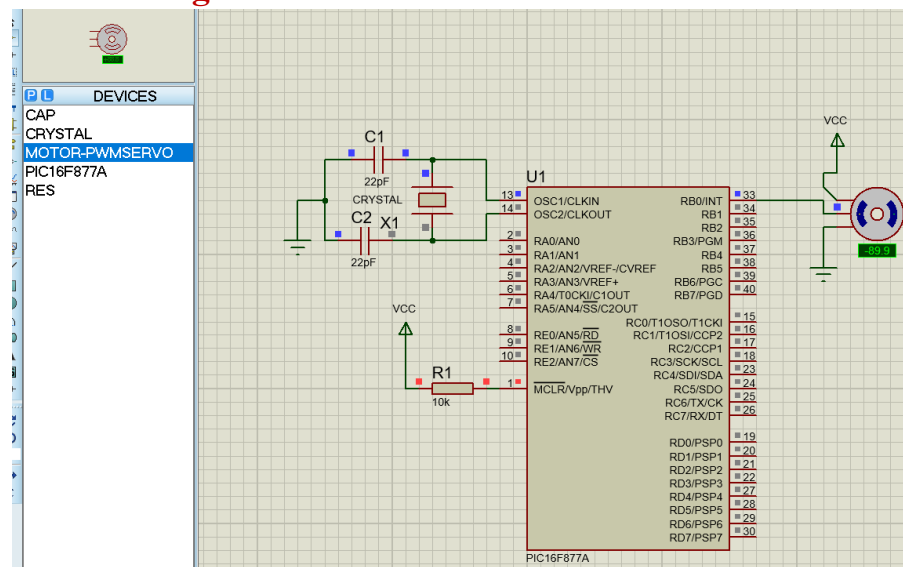


Figure-7: Servo motor using PIC microcontroller

Source Code :

```
unsigned int adc_value;
char txt[7];
void main() {
    ADCON1 = 0x80;    // AN0 analog
    TRISA = 0xFF;     // PORTA input (AN0 input)
    TRISC = 0x80;     // RC7 input, RC6 output
    UART1_Init(9600); // UART 9600 baud
    Delay_ms(100);
    while(1)
    {
        adc_value = ADC_Read(0);
        IntToStr(adc_value, txt); // Integer ? String
        UART1_Write_Text("ADC Value = ");
        UART1_Write_Text(txt);
        UART1_Write(13); // Carriage return
        UART1_Write(10); // New line
        Delay_ms(500);
    }
}
```

Circuit Diagram:(ADC)

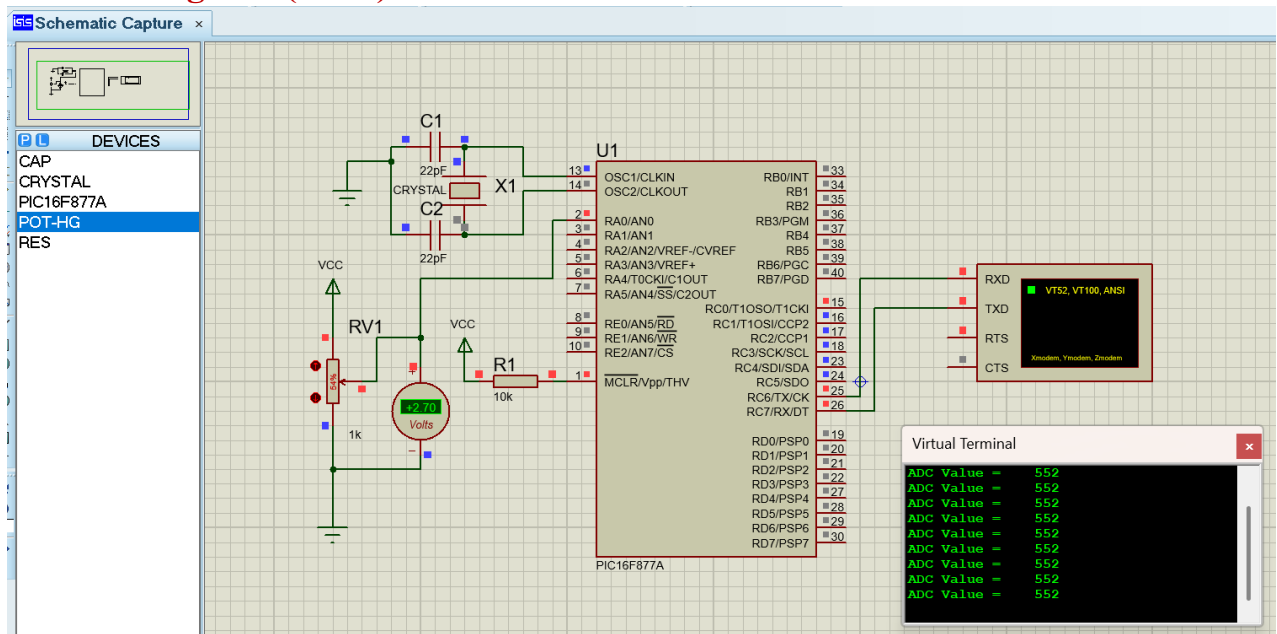


Figure-8: Analog to Digital (ADC) conversion feature of PIC microcontroller.

Source Code :

```
// LCD module connections
sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB2_bit;
sbit LCD_D5 at RB3_bit;
sbit LCD_D6 at RB4_bit;
sbit LCD_D7 at RB5_bit;

sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
// End LCD module connections

char txt1[] = "Information & Communication Engineering";
char txt2[] = "ICE(PUST)";

char i;                // Loop variable

void Move_Delay() {      // Function used for text moving
    Delay_ms(300);       // You can change the moving speed here
}

void main(){
    ADCON1 = 0x06;        // Configure AN pins as digital I/O
    CMCON = 0x07;

    Lcd_Init();
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Cmd(_LCD_CURSOR_OFF);

    Lcd_Out(1,1,txt1);
    Lcd_Out(2,5,txt2);

    Delay_ms(1000);

    while(1) {            // Endless loop
        for(i=0; i<24; i++) {
            Lcd_Cmd(_LCD_SHIFT_LEFT);
            Move_Delay();
        }
    }
```

```

/* for(i=0; i<8; i++) {          // Move text to the right 7 times
    Lcd_Cmd(_LCD_SHIFT_RIGHT);
    Move_Delay();
} */
}
}

```

Circuit Diagram:(LCD)

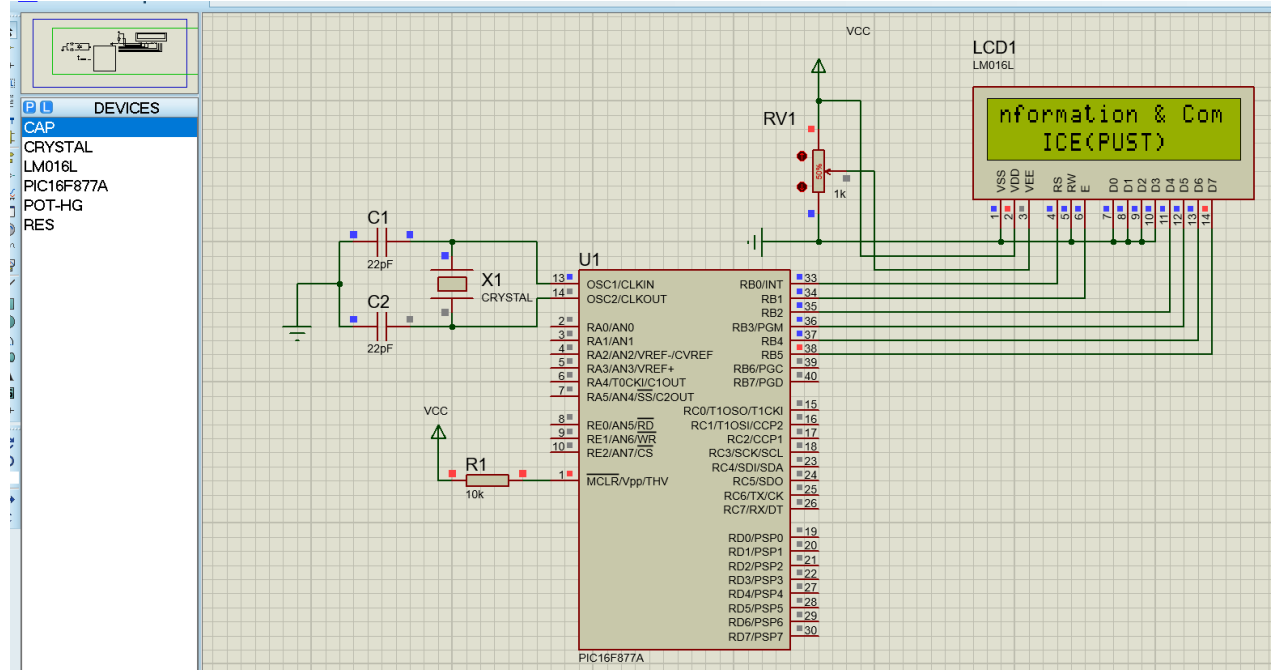


Figure-9: Interface and Display Data on LCD using PIC Microcontroller.

Source Code :

```
// LCD module connections
sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB2_bit;
sbit LCD_D5 at RB3_bit;
sbit LCD_D6 at RB4_bit;
sbit LCD_D7 at RB5_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;

sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
// End LCD module connections
char display[16]="";
void main()
{
    unsigned int result;
    float volt,temp;
    trisb=0x00;
    trisa=0xff;
    adcon1=0x80;
    lcd_init();
    lcd_cmd(_lcd_clear);
    lcd_cmd(_LCD_CURSOR_OFF);
    while(1)
    {
        result=adc_read(0);
        volt=result*4.88;
        temp=volt/10;
        lcd_out(1,1,"Temp = ");
        floattostr(temp,display);
        lcd_out_cp(display);
        lcd_chr(1,16,223); //print at pos(row=1,col=13) "" =223 =0xdf
        lcd_out_cp(" C"); //celcius
    }
}
```

Circuit Diagram:(Temperature sensor)

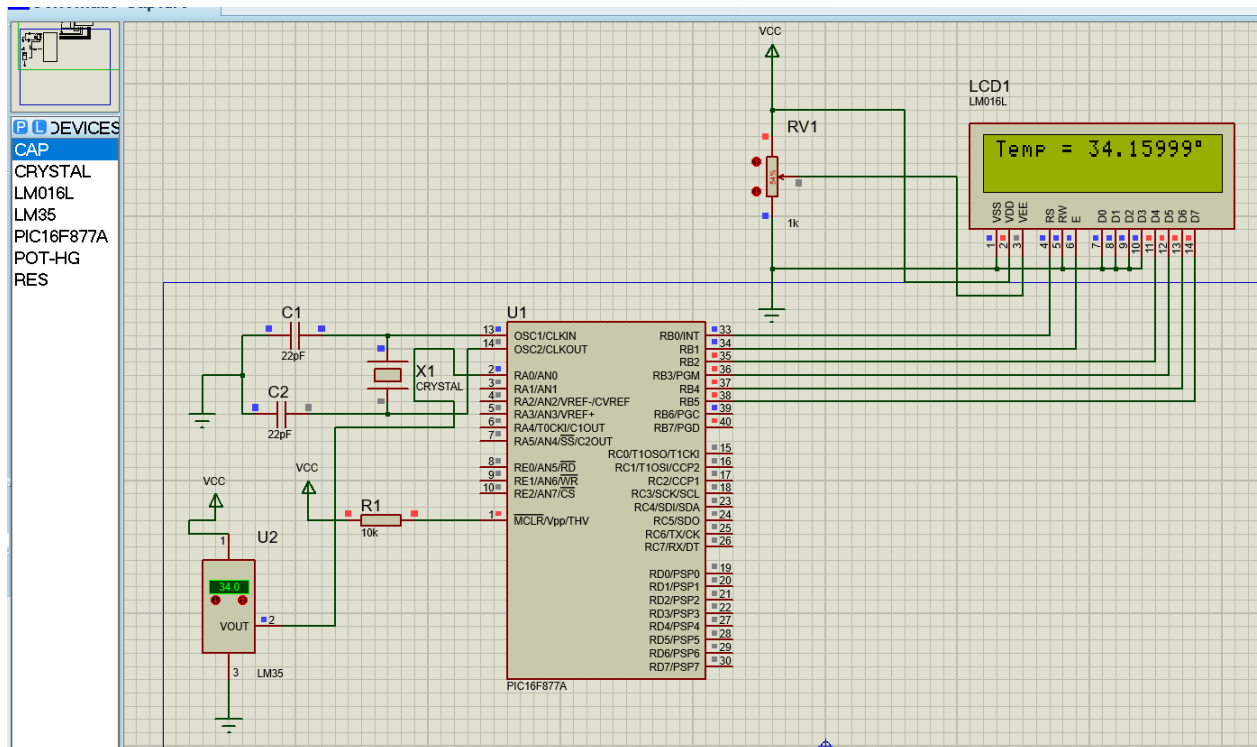


Figure-10: sensor input from LM35 temperature sensor and display corresponding sensor value through LCD display.

Source Code :

```
void main()
{
    TRISD = 0x00;
    PORTD = 0x00;
    do
    {
        PORTD = 0b00000011;
        Delay_ms(500);
        PORTD = 0b00000110;
        Delay_ms(500);
        PORTD = 0b00001100;
        Delay_ms(500);
        PORTD = 0b00001001;
        Delay_ms(500);
    } while(1);
}
```

Circuit Diagram:(Stepper Motor)

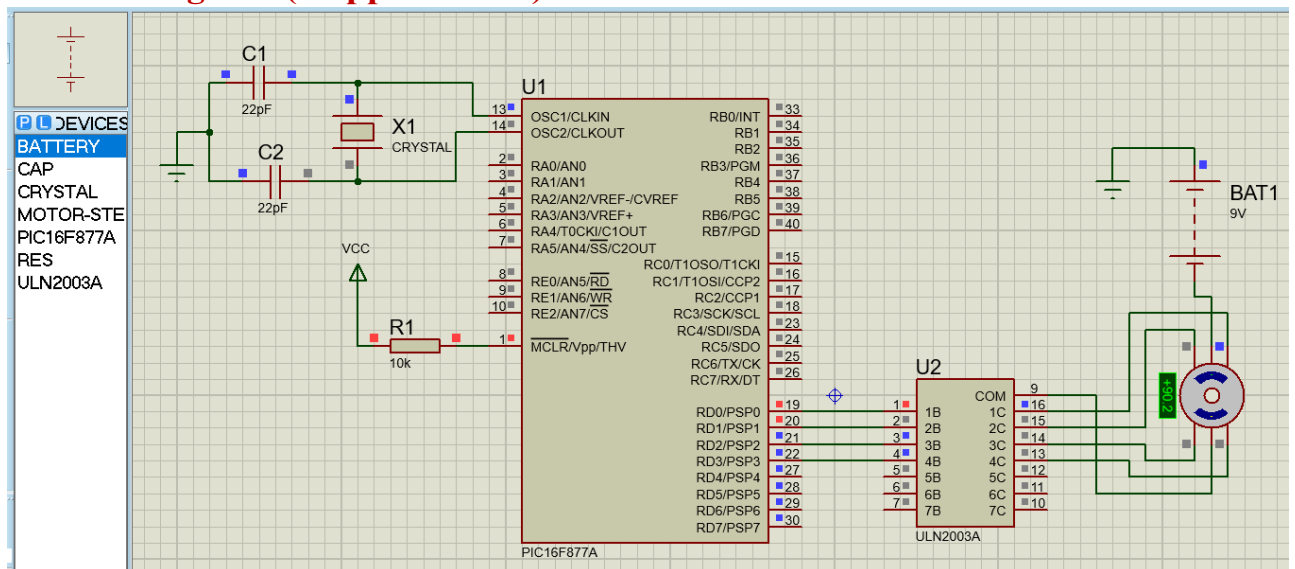


Figure-11: Stepper motor using PIC microcontroller.

Source Code :

```
unsigned char count;
unsigned char seg[] = {0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F};
void main(){
    ADCON1 = 0x06;
    TRISB = 0x00;
    count = EEPROM_Read(0x00);
    if(count > 9) count = 0;
    while(1){
        PORTB = seg[count];
        Delay_ms(1000);
        count++;
        if(count > 9) count = 0;
        EEPROM_Write(0x00, count);
        Delay_ms(20);
    }
}
```

Circuit Diagram:(EEPROM)

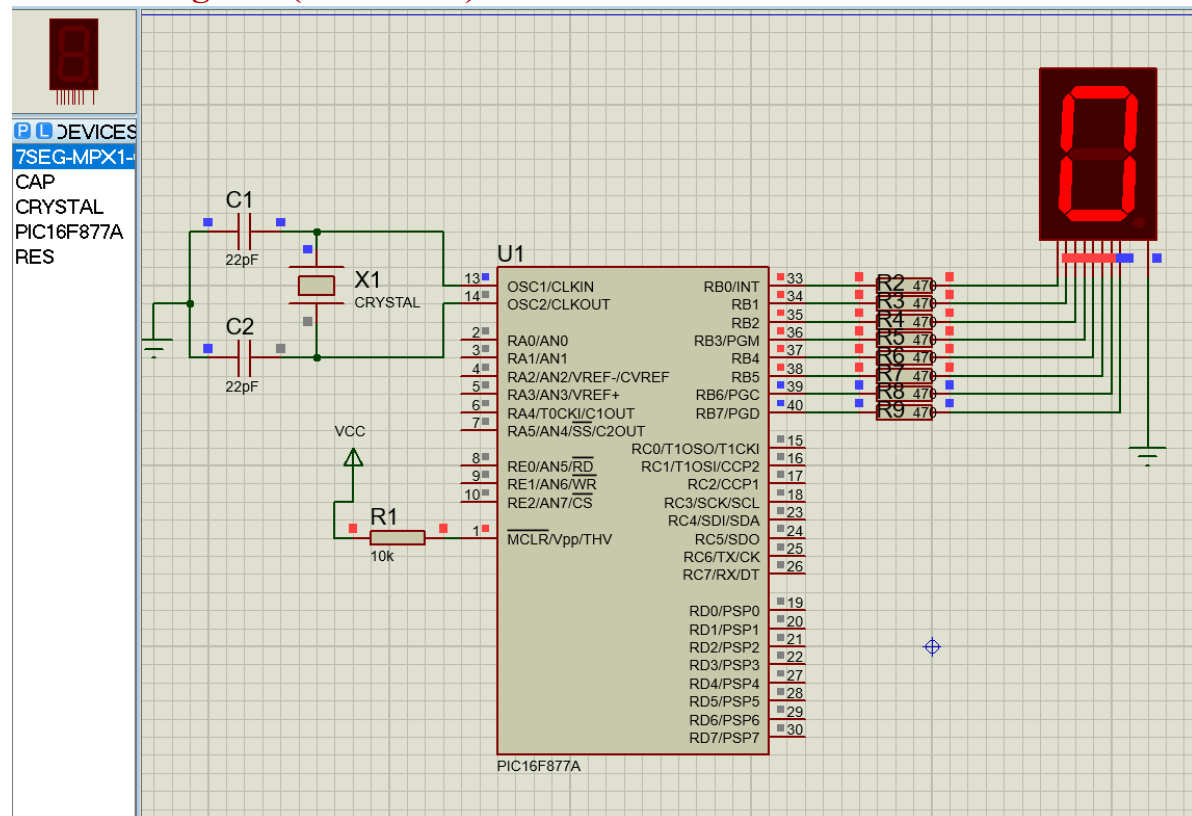


Figure-12: Interface EEPROM for Data Storage and Retrieval using PIC Microcontroller

Source Code :

```
void MSDelay(unsigned char Time){
unsigned char y,z;
for(y=0x00; y<Time; y++)
for(z=0x00; z<0x14; z++);}

void main(){
TRISC = 0x00;
TRISD = 0x00;
while(0x01) {
PORTD = 0x80;
PORTC = 0xFF;
MSDelay(0x0A);
PORTD = 0x40;
PORTC = 0xFF;
MSDelay(0x0A);
PORTD = 0x20;
PORTC = 0x3C;
MSDelay(0x0A);
PORTD = 0x10;
PORTC = 0x66;
MSDelay(0x0A);
PORTD = 0x08;
PORTC = 0xC3;
MSDelay(0x0A);
PORTD = 0x04;
PORTC = 0x81;
MSDelay(0x0A);
PORTD = 0x02;
PORTC = 0x00;
MSDelay(0x0A);
PORTD = 0x01;
PORTC = 0x00;
MSDelay(0x0A); }}

```

Circuit Diagram:(Dot Matrix Display)

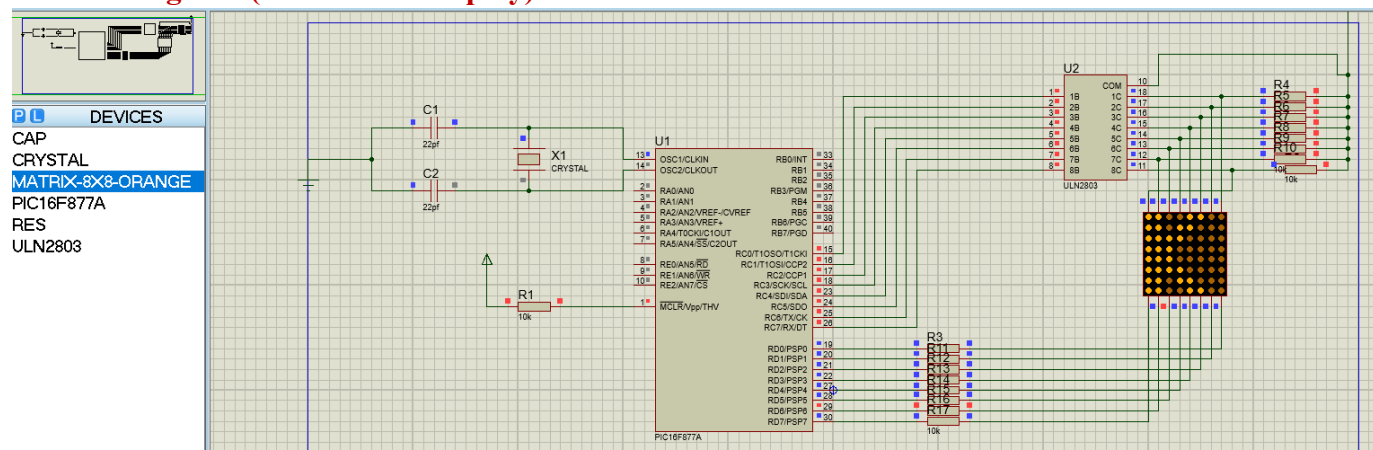


Figure-13: display character through dot matrix display.

